

Windows as Comic Panels: An Art Tool for Exploring Narrative Playmaking on the Digital Screen

*CPSC4900 Fall 2025 Senior Project
Final Report*

Kai Zhang
kai.zhang.kz323@yale.edu
Davenport 2026, Computing and the Arts

Advised By
Prof. Holly Rushmeier
holly.rushmeier@yale.edu
John C. Malone Professor of Computer Science, Yale University

*A Senior Thesis is a partial fulfillment of the
requirements for the Bachelor of Science in
Computing and the Arts*

Department of Computer Science
Yale University
December 1, 2025

Contents

1. Introduction.....	3
2. Background.....	3
2.1 Richard McGuire’s Here and Redefining Panel Composition.....	3
2.2 Nathalie Lawhead on Niche and Experimental Art Tools.....	4
2.3 Chia Amisola’s We Are Only Moving Towards Each Other.....	4
3. Project Goals and Planned Approaches.....	5
3.1 Project Goals.....	5
3.2 Planned Approaches.....	5
4. Implementation.....	5
4.1 Wireframing Storyboard Canvas Editor.....	5
4.2 Tools for Browser Creation, Interaction, and Play.....	6
4.3 Panel Jam: Latest User Interface Version.....	9
4.5 Technical Framework.....	10
4.5 Github Repository.....	12
5. Challenges.....	12
5.1 Feedback Testing.....	12
5.2 Present Issues.....	12
6. Narrative Experimentation.....	13
6.1 Methods of Storytelling.....	13
6.2 Personal Example.....	13
7. Next Steps and Further Exploration.....	14
8. Conclusion.....	14
9. References.....	14

1. Introduction

This project takes inspiration from my fascination with comics and graphic novels, specifically the composition of panels and framing used in the medium. One of my favorite graphic novels is the renowned book titled *Here* by Richard McGuire. The groundbreaking work reinvented how time and space was represented in comics. With the entire story taking place in a single location and making time the subject matter, we are shown the prehistoric past and distant future of a single room. Multiple moments from different years appear in the same panel, sometimes with inset “windows” showing other times layered on top. It revolutionized the grammar of comics storytelling by treating the page as a temporal collage rather than a straightforward sequence of actions.

Taking inspiration, I believe that McGuire’s innovative composition of panels can be applied technically and artistically through pop-up windows on a computer screen. Given the unique environment that the computer screen has, the possibility of storytelling through browser tabs and dialog boxes can be endless. From experimenting with the timing of when pop-ups appear to adjusting how large their sizes are to randomizing the location of where they show up, we have a breadth of options on how to tell a story as we imagine the screen as our comics page. There are numerous net art pieces that already utilize this kind of storytelling, such as the artist Chia Amisola’s project titled *We Are Only Moving Towards Each Other*, a hypertextual narrative on intimacy and love that exists beyond a single tab.

Ultimately, my goal is to develop this medium of computer storytelling as an art tool for people to create comics on their screens. Users can storyboard and experiment with paneling through a Canva-like interface, drawing out their compositions to then later be displayed as windows on their desktop. In addition, users can alter the timing of when those boxes appear, along with their sizes and location to maximize their creative freedom. The UI environment will lean towards an expressive and approachable form for anyone to dive into easily and playfully. This theme of experimental art tool is informed by the written works of software creator and net artist Nathalie Lawhead who talk about building interesting environments that allow users to imagine their own creativity and engage in accessible playmaking.

2. Background

2.1 Richard McGuire’s *Here* and Redefining Panel Composition

Graphic novelist Richard McGuire’s titled *Here* redefined the form that panel composition could take on as a storytelling medium. It changed how time and space were visualized on the page. The story features the corner of a single room jumping through

time with all past, present, and future moments sharing the same physical space. To depict that, smaller inset panels are layered on top of each other showing different time periods. *Here* replaces a linear narrative with a spacial one with time fragment and nonsequential with the “narrative” emerging from spatial juxtaposition, rather than from plot or characters. This is essentially the basis of interactive fiction, a genre of storytelling built around hyperlinks connecting text fragments (i.e. *afternoon, a story* by Michael Joyce (1987)) where the reader navigates by clicking through links and piecing together a narrative.

This kind of storytelling is the experiment that this comic panel storytelling tool tries to replicate. Rather than solely focusing on sequential and linear narratives, the use of this application provides a novel way of approaching storytelling, one that uses the digital interface of the screen and the room it provides for exploratory and unique meaning-making.

2.2 Nathalie Lawhead on Niche and Experimental Art Tools

Net artist Nathalie Lawhead (they/them) writes in their article, “Start making tools! (on making experimental art tools and why it’s valuable for indie game devs to make them),” that quirky and experimental tools can be incredibly valuable for people to use. In their own work *Electric Zine Maker*, it was released as a “toy tool” for creating zines with playful, glitchy, and experimental features that seemed niche and specific, but instead, many found that it provided a meaningful outlet for play and creation. The field of indie art tools is one that trades polish for playfulness, discovery, and surprise, which is in contrast to the many professional creative software out there such as the Adobe Creative Suite. “Professional” tools substitute the weird, wacky, and odd with workflow optimization and efficient mundanity. As I work to develop this art tool, I want to explore these niche modes of expression and dive deeper into nonstandard artistic modes of creating and compelling environments for unabashed making.

2.3 Chia Amisola’s *We Are Only Moving Towards Each Other*

Yale graduate Chia Amisola is an inspiration to the development of this project. Her experiential browser-based narrative *We Are Only Moving Towards Each Other* uncovers themes of digital intimacy, connective care, and dreamlike states that is represented by opening up multiple browser windows for the user to open, close, pull together, and move apart. These windows as such become the narrative fragments distributed as texts, images, and objects for the viewer to navigate through. The viewer’s gestures in moving, closing, and reconfiguring the windows become part of the experience as they influence how the story is perceived and told.

Amisola's work provides an ideal model for the possibilities that this art tool carries. Users are encouraged to imagine in unusual ways and discover new methods of narrative creation.

3. Project Goals and Planned Approaches

3.1 Project Goals

There are three primary goals with this project, which are:

1. Design a canvas editor that allows users to experimentally play with comic panel storytelling on their personal computer.
2. Explore the variety of creative art tools available (i.e. *Kid Pix*, *Tux Paint*) so as to learn how they encourage messy, spontaneous, and genuine art making.
3. Create my own narrative using this browser panel art tool.

3.2 Planned Approaches

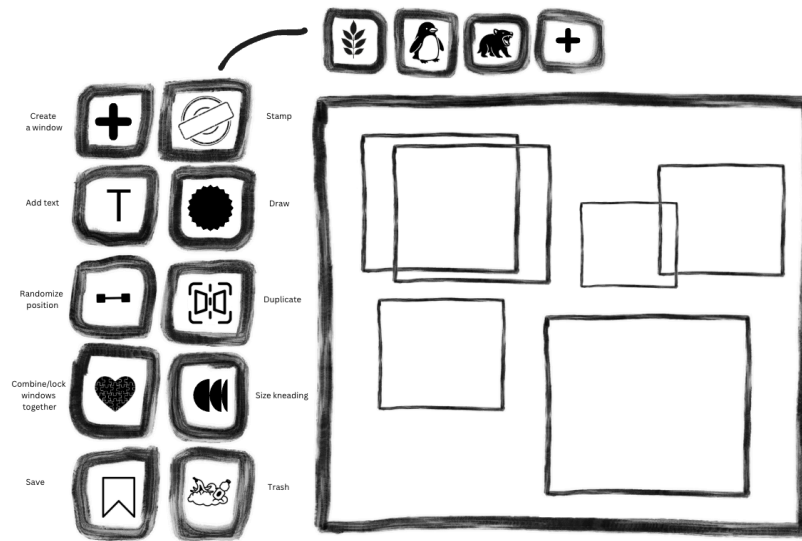
To create this interactive desktop storytelling engine, I'm planning to incorporate the software framework Electron with Javascript and Node.js into my workflow for cross-platform desktop applications. The framework will allow me to implement the logic portion of the project such as opening, sizing, timing, and moving the windows. For the storyboard editor, I plan to use the Fabric.js library to create a drag-and-drop environment for the panel layout. Another aspect will be building the content management system that will allow for adding images created by the user and elements such as panel background, hyperlinks, interactive properties, etc.

4. Implementation

4.1 Wireframing Storyboard Canvas Editor

As I started out my initial design for this browser-window-based storytelling art tool, I took inspiration from two tools that provide a simple and kid-friendly interface for art making – *Kid Pix* and *Tux Paint*. After testing out those software, there were a couple main themes that I took away from using them. One was the amateurism of the software themselves. They didn't feel expertly designed as text felt stretched and alignment of elements weren't exactly precise. Another observation was the playful gestures exhibited throughout the experience of using these tools. Both *Kid Pix* and *Tux Paint* change the mouse cursor to either a pencil or paint brush, respectively, and in the case of *Tux Paint*, it has a penguin mascot that gently guides the user on how to use a tool or adorably cheers for them as they make an artistic mark on the canvas. Additionally, there was an abundance of art-making tools ranging from brushes, stamps, text, and shapes that allowed for an infinite kind of freedom to experiment and mess around.

In the spirit of those themes, I created a simple illustration detailing the layout and properties that this art tool would support. Below is a drawing of my initial idea of what the canvas editor would entail.



Initial drawing of canvas storyboard editor

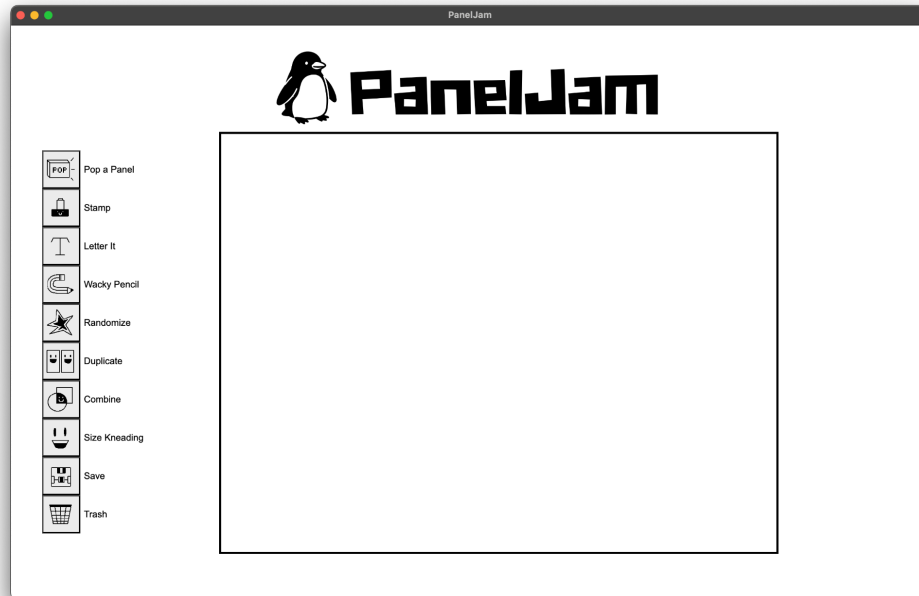
4.2 Tools for Browser Creation, Interaction, and Play

In the digital environment of browser windows, there are a number of ways of users can interact with them. Below is a non-exhaustive list of the interactions that I've discovered, along with possibilities for how users might play with them:

- Moving windows throughout the screen
- Resizing the width and height of a window
- Creating a new browser window
- Duplicating windows
- Making a window the child of a parent window
- Having windows randomly placed throughout the screen
- Arranging windows side-by-side or grid-style, snapping next to each other
- Clicking a window to activate a duplicate window

Being mindful of the timeline and feasibility of this project, I've decided to only implement a couple of the major interactive features such as moving, duplicating, and resizing, but the development of these functions might change depending on what would allow for more experimentation and spontaneous generation of art.

For the visual language of these features, I was inspired to be silly and wanted to incorporate a sense of delightful surprise in their naming and visuals. I made a protoypical interface for the tool which I titled as *Panel Jam*, and below is the initial mockup from October 16, 2025.



Panel Jam's mockup user interface

In a separate course, after listening to a talk from past Yale CPAR student Chia Amisola, I began to rethink the tools that I was planning to implement. The guest talk led me to reconsider how I can make each tool as meaningful as it can be, taking into account the broader possibilities that this digital interface could provide.

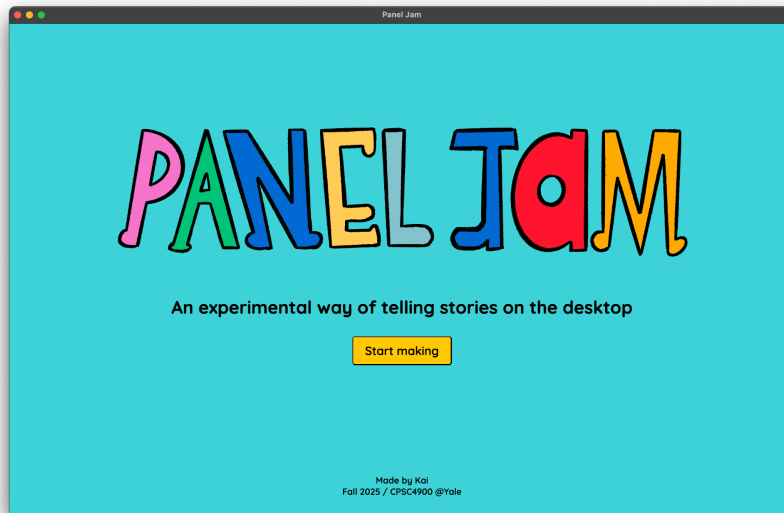
The tools for this window-based storytelling device eventually came down to these options:

<i>Pop a Panel</i>	Creates a window panel on the screen
<i>Stamp</i>	Offers a curated set of panels for user to stamp with on the canvas <i>Stamp</i> includes: ● Frog ● Mouse ● Turtle ● House
<i>Letter It</i>	Inserts text on the window panels

<i>Wacky Pencil</i>	Provides a selection of drawing tools to use <i>Wacky Pencil</i> includes: ● Pencil ● Spraypaint ● Line ● Crayon ● Marker ● Ink
<i>Layout</i>	Gives preset layouts of panels for users to choose from <i>Layout</i> includes: ● 1x2 ● 2x2 ● 4x4 ● Circle ● Wave ● Random
<i>Background</i>	Offers users to choose what kind of background they want to have <i>Background</i> includes: ● White ● Black ● Image ● Video
<i>Surprise</i>	Surprises users with a random arrangement of panels on their desktop
<i>Animate</i>	Animates the movements of panels <i>Animate</i> includes: ● Move Right ● Move Left
<i>Save</i>	Saves objects on the screen to a gallery
<i>Gallery</i>	Allows users to see previously saved canvas pieces
<i>Preview</i>	Transforms objects on the canvas to panels that users can see on their screen
<i>Clear Preview</i>	Clear objects on the canvas and panels on user's desktop

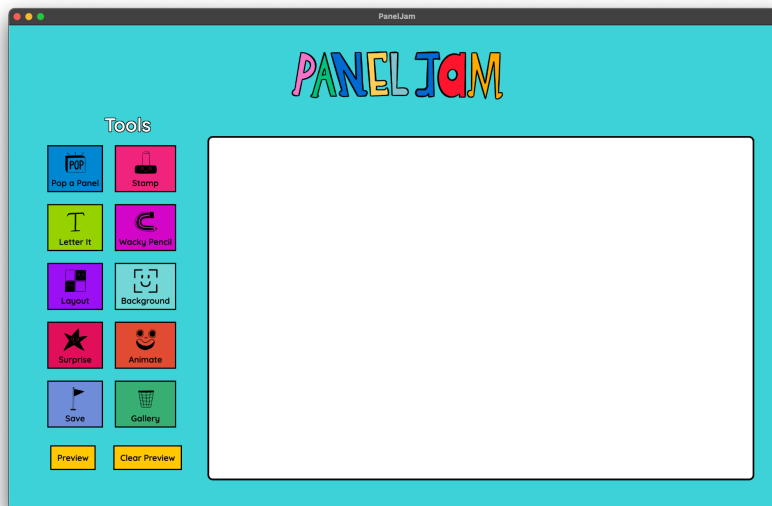
4.3 Panel Jam: Latest User Interface Version

The most recent version of the interface for this art tool can be seen below, where the user is first greeted with a home page titled *Panel Jam* and a description that reads “an experimental way of telling stories on the desktop.”



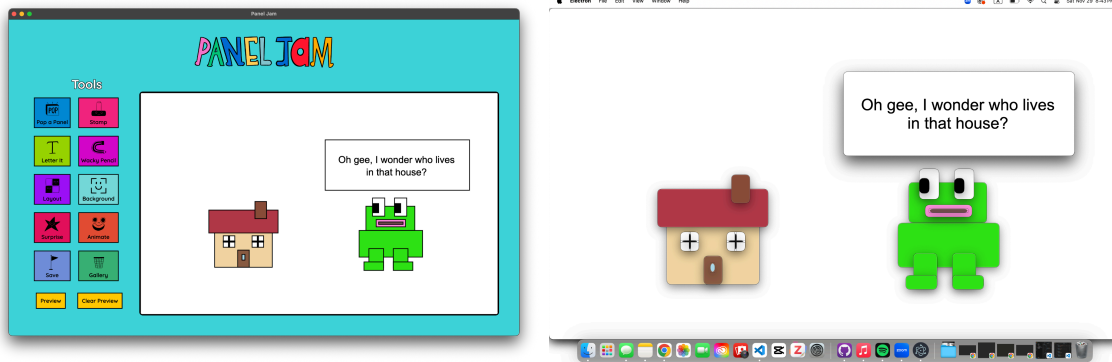
Panel Jam's home page

After the user clicks “Start making,” they are shown a revised interface that emphasizes a friendly and dynamic look with popping colors and hand-drawn images. The left side showcases a grid of tools that the user can play with, and the right side is the canvas editor that will contain all of the texts, rectangles, and objects.



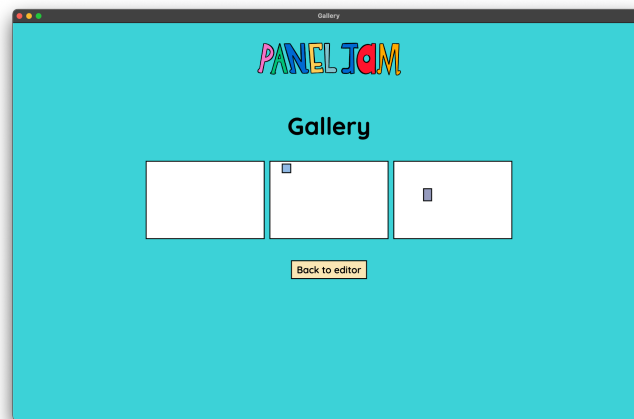
Panel Jam's canvas editor layout

Once the user is satisfied with what they’ve made, they can press *Preview* which displays the objects on the canvas editor as panels on the desktop screen, as seen below.



Example of work being previewed

To save, users can press *Save*, which will send them to a gallery view of previously saved works. Users can click to reopen those pieces on their canvas editor or delete them to make room for new stories.



Panel Jam's gallery view

4.5 Technical Framework

This project's technical setup uses the Electron framework for creating a desktop app with web-based tools, combined with Fabric.js to power the canvas editing capabilities.

The Electron framework is composed of two types of processes: main and renderer. An Electron app has a single main process which is the application's entry point and management system, creating and running windows using the BrowserWindow module. The renderer process is thus responsible for rendering the web content to be displayed on the windows. Each BrowserWindow has its own separate renderer processes that is

running simultaneously. Code that is run in renderer processes behave according to web standards which involves HTML, CSS, and Javascript.

Main Process

The main process is found in `main.js` which controls the application lifecycle and manages all the Electron BrowserWindows. It creates the main application window, loads HTML files onto the windows, spawns the preview pop-up panels, listens for IPC (Inter-Process Communication) messages sent from the renderer using `ipcMain.on`, and sends webContents to be previewed using BrowserWindow APIs. IPC messages allow for data and control information to be passed between separate processes, enabling cross-platform communication and modular component synchronization. This is done with `preload.js` which bridges the communication between renderer and main process.

Renderer Process

The renderer process is what renders the web content that is processed onto the browser windows. It includes the front-end components: the editor UI, Fabric.js canvas, and tool logic.

The `index.html` is the main UI page of the application. It contains the toolbar buttons, canvas container for Fabric.js editor, script imports, and styling sheets. The two stylesheets found in `index.html` are `styles.css` and `tools.css` which define the layout and appearance of the interface from layout to colors.

The renderer logic is controlled by `script.js` which implements all tools and user interactions. It contains tool logic for: free drawing, adding text and editing properties, setting up panel layouts, inserting preset shapes, saving objects to LocalStorage, sending canvas data to preview windows via IPC, along with other behavioral capabilities. A couple of brush options are credited to the [Fabric.js add-on](#) built by Arjan Haverkamp. What is loaded onto the preview browser windows is from `content.html` which acts as the template page for objects to be displayed on to. The page then loads texts and brush paths using ElectronAPI and IPC listener wrapper onto the body tag.

Lastly, the logic for saving canvas drawings is done by serializing canvas objects to JSON which is an open file format that saves all objects on a canvas to be imported, exported, and stored. The canvas JSON is received by `gallery.html` to then be parsed and rendered.

4.5 Github Repository

The current Github repository of the code for this project can be found [here](#).

5. Challenges

One of the initial challenges in setting up this project was learning how to navigate the Electron workspace. Setting up the main and renderer processes took some time, but once I understood how to structure both environments, the workflow became much smoother. The challenges then switched to finely positioning elements, scaling between canvas and screen size, and implementing the logic behind the creation tools.

Midway through the project, a larger question loomed over me which was “How can we maximize the potential of the desktop to inspire users on what they can create?” This made me recognize my own shortcomings of what I can do on the desktop and thus pushed me to imagine more critically about what aspects of this space that I can better leverage. Aside from the conspicuous capabilities such as inserting browser windows and changing screen sizes, I was considering how I might incorporate more dynamism and movement in this tool. Those ideas lead me to make an attempt at animating the windows and accepting user uploaded content.

5.1 Feedback Testing

A major part of the project was to focus on user testing and feedback. With any tool, there are many unseen ways someone might interact with it. Testing this tool became especially important, as it could reveal new ways user might engage with it uncover previously hidden bugs, and highlight areas that could improve the user experience. I was able to gather a couple of testers to provide feedback, and the initial feedback was about improving the intuitive feel of the tools, providing demos for how to use certain features, and adding more styling to the brushes and text.

After the initial round, I gave the project to a graphic design professor at the Yale School of Art to try who offered ideas about accentuating the accidents and bugs and considering ways that an interaction might lead, disrupt, or force another action on the screen.

5.2 Present Issues

Currently, there are a few outstanding bugs that the project faces. The animate tool, although showcases movement on the canvas, doesn't fully function when previewed. The issue likely stems from improperly packaging position information and sending it over to the Electron main process. Another issue is cleaning up the preview function. Certain brush tools, because they act as images and not paths, are previewed as images, when they instead should be previewed only when contained by a rect panel.

There are additional minor problems present within the system including gallery thumbnails not accurately displayed, panels jumbled up when placed outside of the

canvas frame, and the layering of objects not being so intuitive. As I expose the tool to more testing, there will inevitably be more issues that arise, but for the most part, there is a baseline functionality that creates room for both prompted and spontaneous play.

6. Narrative Experimentation

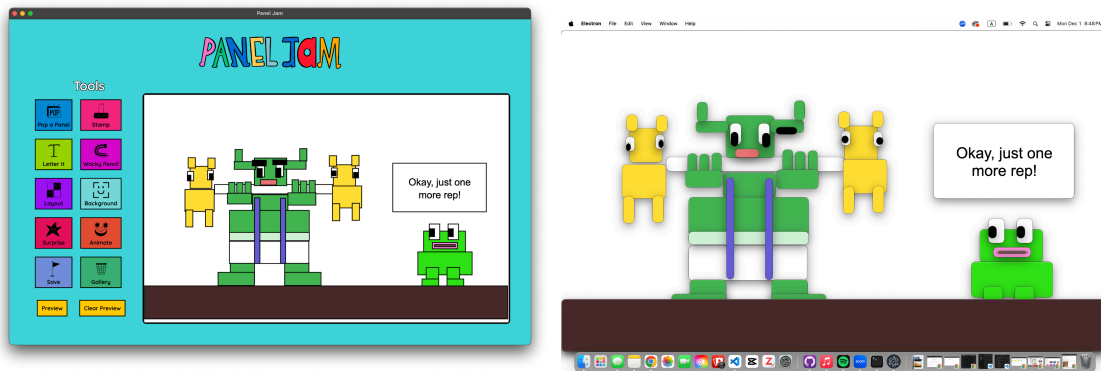
6.1 Methods of Storytelling

This art platform facilitates a range of engagement methods for storytelling, experimentation, and making. The layout feature provides traditional formats such as a 4-panel grid, which creators can draw on to craft simple narratives and short sequences of events for key actions and punchy delivery. The platform also allows for people to design characters using the panel feature and customize the panel colors. Moreover, users can create intricate patterns made up of windows, compose a panel sketch series, or render a maximal display of texts, rects, and objects.

An ideal scenario for this application is that given the constraints of the workspace with its broad, but still limited, set of tools, people can still find a way to engage in a form of making that wasn't previously imagined. I'm interested in how users can transform this tool to explore their own ideas and adapt it to their interests and needs.

6.2 Personal Example

The way that I personally like to interact with this program is shaping characters using the panel tool and creating a scene with them. Here is an example below.



7. Next Steps and Further Exploration

The next steps of this project involves fine-tuning the features and its performance. While there are numerous add-ons I could incorporate into the work, with some further expanding the space for playful exploration, I believe better time and effort could be

devoted to refining the user experience and enhancing the precision of each tool. There are lingering issues that muddy the canvas, resulting in frustrated arts-making. Providing demos for how to use the tools and showing examples of previous work would also be helpful in priming the user on how to create. Further exploration of this project will focus on how to package these window-based comic narratives to be viewed across devices for others to experience. Questions about how to distribute these stories and where to exhibit them has been a floating idea that deserves more thoughtful attention in the future. Lastly, the themes of this project will carry into spring semester as I develop the art component of my CPAR senior project.

8. Conclusion

Overall, this experimental storytelling tool provides a solid foundation for what I originally envisioned it to be. At the beginning, the three main goals were to design a canvas editor for experimenting with comic panel storytelling, to explore how certain tools foster spontaneous art creation, and to create my own narratives using what I have developed. This senior project has turned out to be a fruitful investigation of what the desktop environment has to offer and sharing such discoveries with those who have seen or played around with this program. There is still much that could be improved, added, and changed in the application, but at this stage, it already presents many avenues for creative exploration. With an array of unexplored functionalities and hidden interfaces, I still have a lot to discover in this digital jungle that is my desktop.

9. References

- [1] Amisola, Chia. *We Are Only Moving Towards Each Other*. When We Love series, Everything I Love / artist's website, <https://everythingi.love/world/whenwelove/moving.html>.
- [2] Lawhead, Nathalie. "Start Making Tools! (On Making Experimental Art Tools and Why It's Valuable for Indie Game Devs to Make Them)." *Candybox Blog*, 18 Dec. 2020, nathalielawhead.com/candybox/start-making-tools-on-making-experimental-art-tools-and-why-its-valuable-for-indie-game-devs-to-make-them.
- [3] McGuire, Richard. *Here*. Pantheon Books, 2014